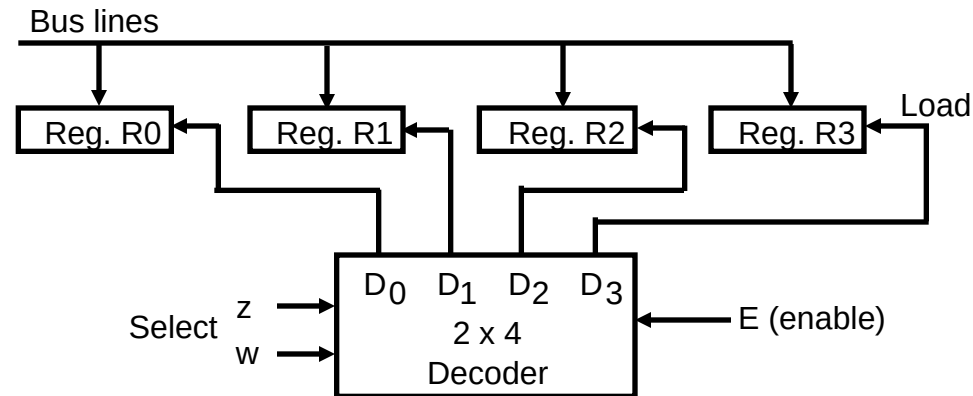
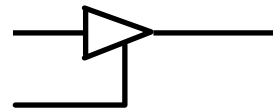


TRANSFER FROM BUS TO A DESTINATION REGISTER



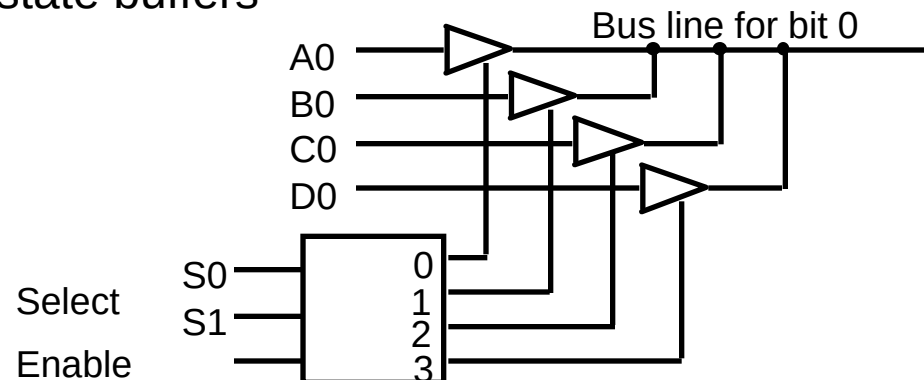
Three-State Bus Buffers

Normal input A
Control input C

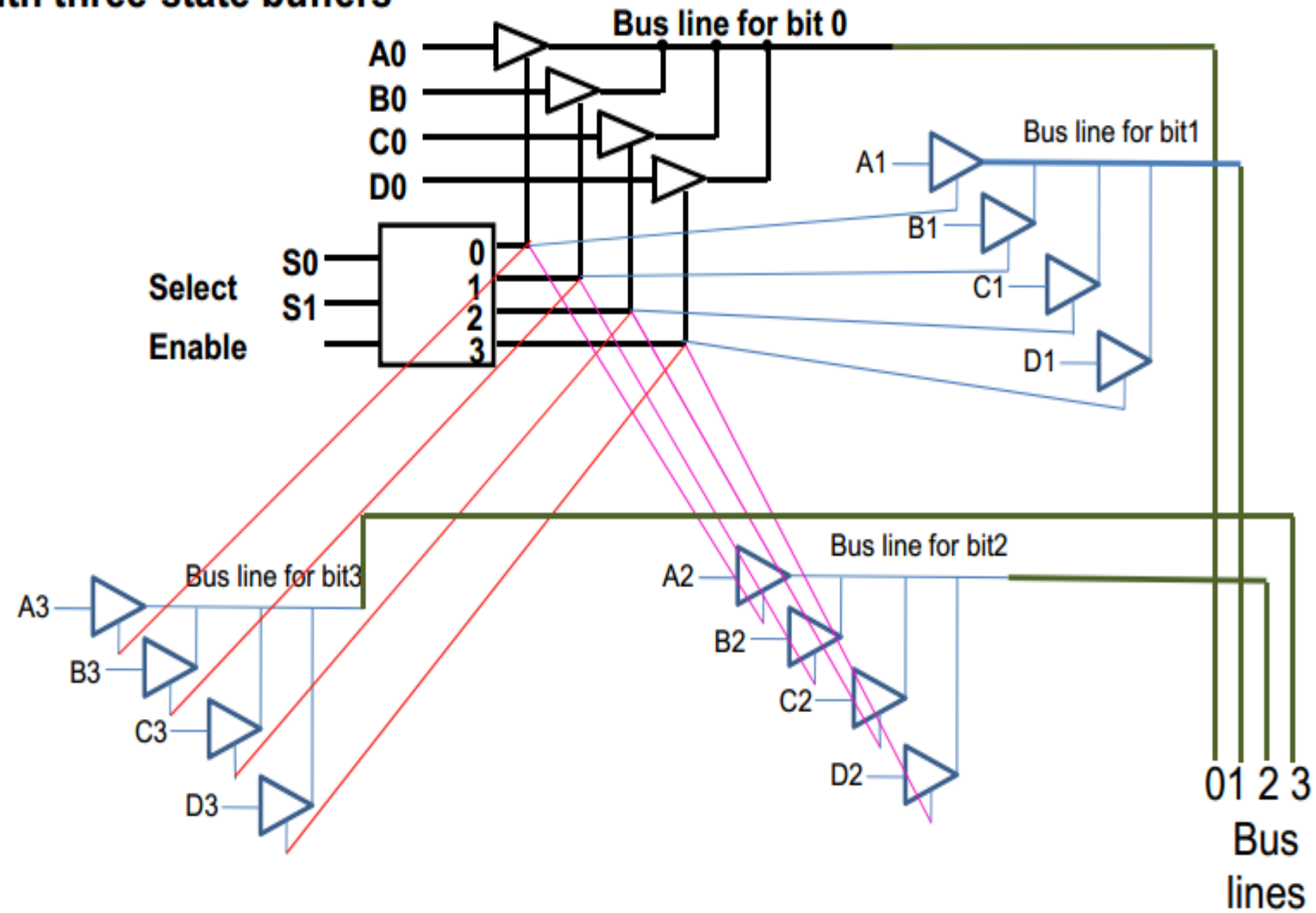


Output $Y=A$ if $C=1$
High-impedance if $C=0$

Bus line with three-state buffers



Bus line with three-state buffers



BUS TRANSFER IN RTL

- Depending on whether the bus is to be mentioned explicitly or not, register transfer can be indicated as either

or $R2 \leftarrow R1$

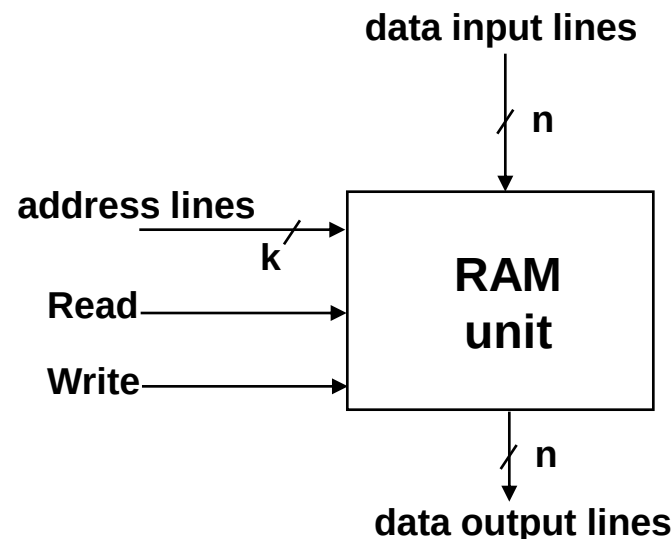
$BUS \leftarrow R1, R2 \leftarrow BUS$

- In the former case the bus is implicit, but in the latter, it is explicitly indicated



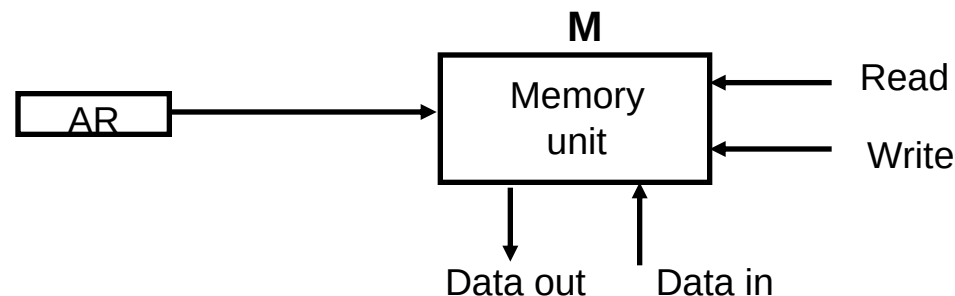
MEMORY (RAM)

- Memory (RAM) can be thought as a sequential circuits containing some number of registers
- These registers hold the *words* of memory
- Each of the r registers is indicated by an *address*
- These addresses range from 0 to $r-1$
- Each register (word) can hold n bits of data
- Assume the RAM contains $r = 2^k$ words. It needs the following
 - n data input lines
 - n data output lines
 - k address lines
 - A Read control line
 - A Write control line



MEMORY TRANSFER

- Collectively, the memory is viewed at the register level as a device, M.
- Since it contains multiple locations, we must specify which address in memory we will be using
- This is done by indexing memory references
- Memory is usually accessed in computer systems by putting the desired address in a special register, the *Memory Address Register* (*MAR*, or *AR*)
- When memory is accessed, the contents of the MAR get sent to the memory unit's address lines



MEMORY READ

- To read a value from a location in memory and load it into a register, the register transfer language notation looks like this:

$$R1 \leftarrow M[AR]$$

- This causes the following to occur
 - The contents of the MAR get sent to the memory address lines
 - A Read (= 1) gets sent to the memory unit
 - The contents of the specified address are put on the memory's output data lines
 - These get sent over the bus to be loaded into register R1



MEMORY WRITE

- To write a value from a register to a location in memory looks like this in register transfer language:

$M[AR] \leftarrow R1$

- This causes the following to occur
 - The contents of the MAR get sent to the memory address lines
 - A Write (= 1) gets sent to the memory unit
 - The values in register R1 get sent over the bus to the data input lines of the memory
 - The values get loaded into the specified address in the memory



SUMMARY OF R. TRANSFER MICROOPERATIONS

$A \leftarrow B$	Transfer content of reg. B into reg. A
$AR \leftarrow DR(LH)$	Transfer content of Lower portion of reg. DR into reg. AR
$A \leftarrow \text{constant}$	Transfer a binary constant into reg. A
$ABUS \leftarrow R1,$ $R2 \leftarrow ABUS$	Transfer content of R1 into bus A and, at the same time, transfer content of bus A into R2
AR	Address register
DR	Data register
$M[R]$	Memory word specified by reg. R
M	Equivalent to $M[AR]$
$DR \leftarrow M$	Memory <i>read</i> operation: transfers content of memory word specified by AR into DR
$M \leftarrow DR$	Memory <i>write</i> operation: transfers content of DR into memory word specified by AR

